

FitTrack AI - Full Stack Fitness Tracking and AI Recommendation System

The FitTrack AI project is a full-stack fitness tracking application combining a React frontend, a secure Node.js and Express.js REST backend, MongoDB Atlas persistence, JWT authentication, workout management, dashboard analytics, and Google Gemini AI integration. This document follows the supplied FitTrack AI reference document's organization, typography, spacing, and presentation style while documenting the upgraded implementation. It replaces the earlier Postman-oriented testing flow with Thunder Client and documents the React client, dashboard, profile management, pagination, filtering, and current Atlas configuration.

Description

The FitTrack AI backend is a secure, modular, and AI-enabled RESTful application developed to simplify workout tracking and provide intelligent fitness guidance. The backend uses Node.js, Express.js, MongoDB, Mongoose, JWT, bcryptjs, and the Google GenAI SDK.

The upgraded project adds a React client so the REST API can be consumed through a web interface. The backend remains responsible for authentication, workout CRUD operations, searching, dashboard analytics, profile management, validation, authorization, and AI service invocation.

MongoDB Atlas is used as the persistent database. Protected endpoints require a valid JWT token, and passwords are hashed using bcryptjs before storage.

Scenario-Based Case Study

Background

A fitness user wants one application for recording workouts, reviewing progress, searching workout history, and obtaining personalized recommendations. Manual notebooks or spreadsheets make it difficult to maintain a consistent history and calculate useful statistics.

Problem

- Difficulty maintaining a centralized workout history.
- Manual tracking can make historical review slow and inconsistent.
- Personal workout information requires authenticated access.
- Users need search, category filtering, and date filtering.
- Users need a dashboard to summarize workout activity.
- Users may want personalized AI guidance based on fitness information.

Solution

FitTrack AI centralizes workout data behind authenticated REST APIs and presents it through a React interface. Dashboard statistics summarize activity, while the Gemini service can generate recommendations and fitness insights when the Google AI project has generation access.

Solution and Objectives

The main objective is to provide a maintainable full-stack fitness application with clear separation between presentation, API logic, persistence, authentication, analytics, and AI services.

- Secure registration and login.
- JWT-protected profile and workout resources.
- Complete workout create, read, update, and delete operations.
- Search by workout name or category.
- Pagination and category/date filtering.
- Dashboard analytics and recent activity.
- React-based web interface.
- MongoDB Atlas storage.
- Google Gemini recommendation and insight service.

The architecture also leaves room for nutrition tracking, BMI calculation, reminders, wearable integration, and additional fitness analytics.

Usage

User Usage

A registered user creates an account using the registration endpoint and logs in using email and password. After successful authentication, the backend returns a JWT token. The token is supplied in the Authorization header for protected operations.

- Create and manage a profile.
- Create workout records.
- View paginated workout history.
- Search and filter workouts.
- Retrieve, update, and delete a workout.
- View dashboard statistics.
- Request AI workout recommendations.
- Request AI fitness insights.

AI Usage

Workout recommendations accept age, fitness goal, and experience level. Fitness insights accept total workouts, average duration, and total calories burned.

Outcome

The upgraded FitTrack AI provides a complete path from web interface to secured API to database and AI service. The React client consumes standardized JSON responses, while the Express backend keeps authentication, data access, analytics, and AI concerns separated. The application eliminates repetitive manual workout record keeping and provides structured views of workout history and progress. The dashboard reduces the need to calculate totals manually.

The AI integration is implemented in the application. Actual AI generation depends on the Google AI project having permission to generate content for the configured model. If Gemini

access is unavailable, the non-AI authentication, workout, search, and dashboard features remain independent.

1. Software Requirements

- Windows 10/11, macOS, or Linux.
- Node.js with npm.
- Visual Studio Code.
- MongoDB Atlas account and database connection string.
- Thunder Client VS Code extension.
- Google AI project and Gemini API key for AI features.
- Modern web browser.

The client and server are developed as separate npm applications. Internet access is required for MongoDB Atlas and Gemini API communication.

2. Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum; 16 GB recommended for development.
- Storage: At least 2 GB free working space.
- Internet connection for Atlas and Gemini services.

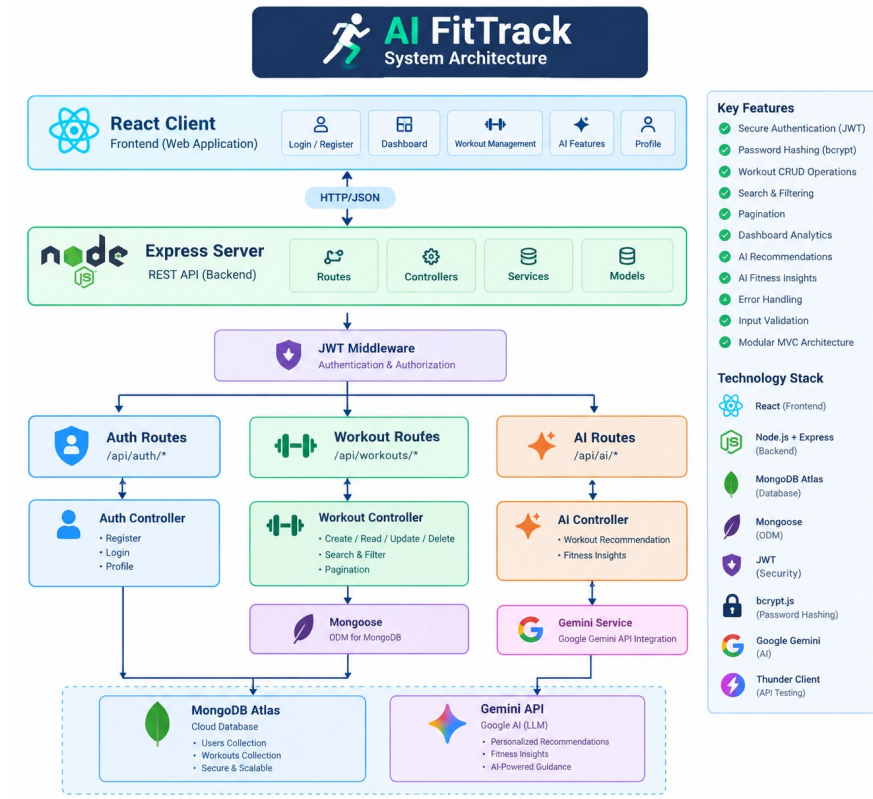
Actual requirements vary depending on the number of browser tabs, development tools, and other applications running simultaneously.

Epic 1: Project Architecture

The project follows a layered architecture. React acts as the client interface, Express provides the HTTP gateway, routes map endpoints to controllers, middleware protects secured resources, services encapsulate reusable integrations, Mongoose models define persistence, and MongoDB Atlas stores application data.

Typical request flow:

```
React / Thunder Client → Express → Route → JWT Middleware → Controller → Model or Service → MongoDB Atlas / Gemini → JSON Response
```



Architecture diagram

Overall Architecture Components

- React client application.
- Express.js API server.
- Authentication and error middleware.
- Authentication, workout, dashboard, and AI routes.
- Authentication, workout, dashboard, and AI controllers.
- Gemini AI service.
- Mongoose User and Workout models.
- MongoDB Atlas database.

Each layer communicates through defined inputs and outputs, reducing coupling and making individual modules easier to test and maintain.

Project Folder Structure

The upgraded project is divided into a React client and an Express server.

```
AI-FitTrack-React-Upgraded/
├── client/
│   ├── index.html
│   ├── package.json
│   └── src/
│       ├── lib/api.js
│       ├── main.jsx
│       └── styles.css
```

```
├── server/
│   ├── .env
│   ├── package.json
│   └── src/
│       ├── app.js
│       ├── server.js
│       ├── config/db.js
│       ├── controllers/
│       ├── middleware/
│       ├── models/
│       ├── routes/
│       └── services/
```

Client Layer - React Frontend

The React client is the presentation layer for the upgraded project. It communicates with the Express backend through HTTP requests and displays authentication, workout, dashboard, and AI information.

- Sends API requests.
- Displays workout information.
- Handles authenticated user flow.
- Displays dashboard analytics.
- Invokes AI recommendation and insight APIs.
- Displays success and error responses.

Express Server Gateway

The Express server is the central gateway. It loads environment variables, configures CORS, security headers, request logging, JSON parsing, routes, and centralized error handling.

The health endpoint is:

```
GET http://localhost:5000/api/health
```

A successful response identifies the API and enabled feature groups.

Authentication Middleware

Protected APIs use JWT authentication middleware. The middleware reads the Authorization header, extracts the bearer token, verifies the signature using JWT_SECRET, identifies the user, and attaches authenticated user information to the request.

```
Authorization: Bearer YOUR_JWT_TOKEN
```

- Prevents unauthorized access.
- Secures personal workout information.
- Allows only authenticated users to access protected resources.
- Keeps authentication logic outside individual controllers.

Route Layer

Routes define the API endpoints and map incoming HTTP requests to controller methods.

Route Group	Base Path	Purpose
Authentication	/api/auth	Registration, login, profile

Workouts	/api/workouts	Workout CRUD and search
Dashboard	/api/dashboard	Analytics and recent activity
AI	/api/ai	Recommendations and fitness insights

All routes except health, registration, and login require authentication as implemented in the server.

Controller Layer

Controllers receive validated requests from routes and coordinate database operations or service calls.

- Authentication Controller: registration, login, profile retrieval and profile update.
- Workout Controller: create, list, search, get by ID, update, delete.
- Dashboard Controller: totals, averages, streak, categories, weekly activity and recent records.
- AI Controller: validation and invocation of Gemini recommendation and insight services.

Service Layer

The service layer contains reusable logic that should not be tightly coupled to HTTP route definitions.

Gemini AI Service

- Prepares prompts.
- Sends model requests.
- Receives generated content.
- Returns formatted recommendations.
- Surfaces provider errors for controlled API handling.

Authentication Support

- bcryptjs hashes and compares passwords.
- jsonwebtoken creates and verifies JWT tokens.

Database Layer - MongoDB Atlas

MongoDB Atlas is the project's persistent NoSQL database. The connection string is supplied through MONGO_URI in the server environment file.

The main collections are Users and Workouts. Mongoose provides schema validation, relationships, timestamps, and database access methods.

The current project does not require a local MongoDB service for normal development because it connects directly to Atlas.

Entity Relationship - User

The User entity stores authentication and fitness profile information.

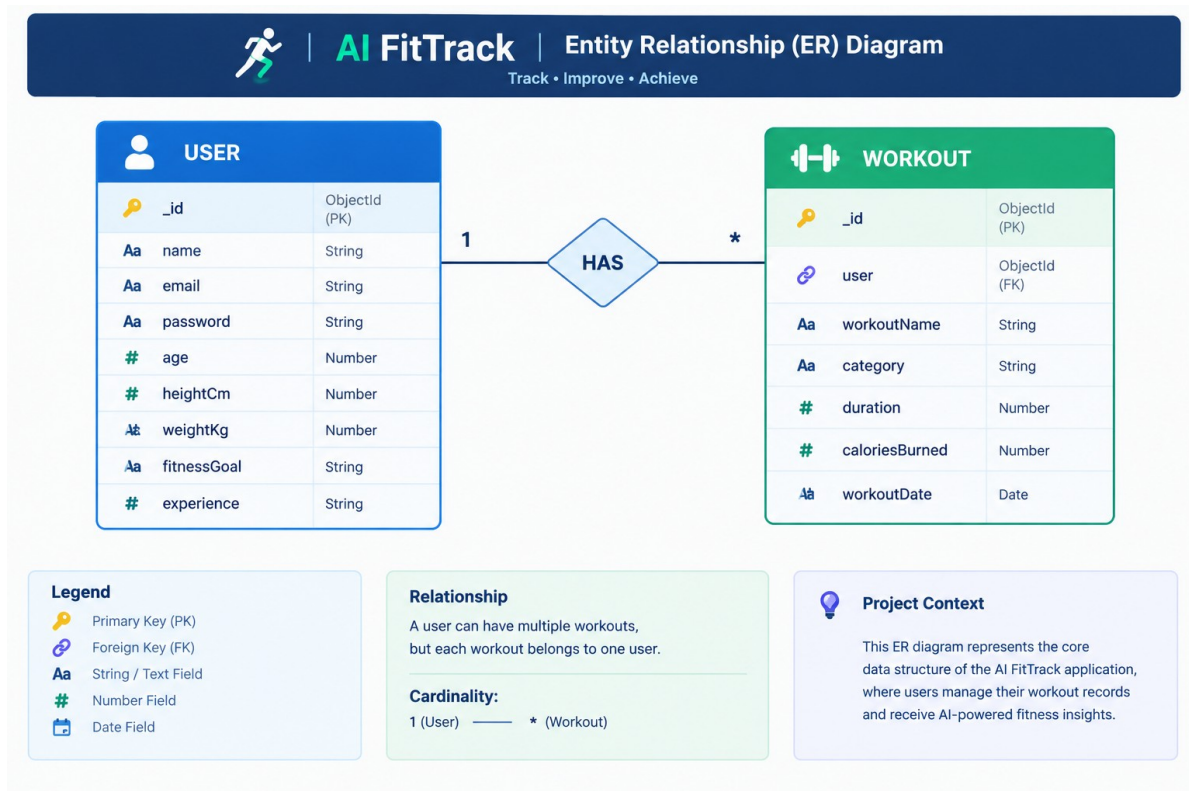
Field	Type	Description
_id	ObjectId	Primary key
name	String	User name
email	String	Unique email address
password	String	Bcrypt-hashed password
age	Number	Profile age
heightCm	Number	Height in centimeters
weightKg	Number	Weight in kilograms
fitnessGoal	String	Selected fitness goal
experience	String	Beginner, Intermediate, or Advanced
createdAt / updatedAt	Date	Automatic timestamps

Entity Relationship - Workout

The Workout entity stores workout information belonging to an authenticated user.

Field	Type	Description
_id	ObjectId	Primary key
user	ObjectId	Reference to User
workoutName	String	Workout name
category	String	Workout category
duration	Number	Duration in minutes
caloriesBurned	Number	Calories burned
workoutDate	Date	Workout date
createdAt / updatedAt	Date	Automatic timestamps

Relationship: one user can have many workout records, while each workout belongs to one authenticated user.



```
    "email": "john@example.com",
    "password": "12345678"
  }
  POST /api/auth/login
  {
    "email": "john@example.com",
    "password": "12345678"
  }
```

Workout Management

Workout management implements create, read, update, and delete operations. Each workout is associated with the authenticated user's ID.

```
POST /api/workouts
{
  "workoutName": "Morning Running",
  "category": "Running",
  "duration": 30,
  "caloriesBurned": 250,
  "workoutDate": "2026-09-24"
}
```

- Allowed categories include Cardio, Strength Training, Yoga, Running, Cycling, and Walking.
- Duration is recorded in minutes.
- Calories burned are stored as a numeric value.
- Workout date is stored as a date value.

Workout Search, Pagination and Filters

The current workout list endpoint supports page, limit, category, from, to, and sort query parameters. Search is available through a dedicated search endpoint.

```
GET /api/workouts?page=1&limit=10
GET /api/workouts?category=Running
GET /api/workouts?from=2026-09-01&to=2026-09-24
GET /api/workouts/search?q=running
GET /api/workouts/search?q=morning&category=Running
```

Dashboard Analytics

GET /api/dashboard returns an authenticated summary of workout activity.

- totalWorkouts
- totalDuration
- totalCalories
- averageDuration
- streak
- categories
- weekly activity for the recent period
- recent workout records

The dashboard reduces manual calculation and gives the React client a single endpoint for summary information.

Google Gemini Integration

The project uses a dedicated Gemini service through the Google GenAI SDK. The model name is configured through GEMINI_MODEL so the provider configuration can be changed without modifying controller logic.

```
GEMINI_MODEL=gemini-3.6-flash
```

Workout Recommendation Inputs

- Age
- Fitness Goal
- Experience Level

Fitness Insight Inputs

- Total Workouts
- Average Duration
- Total Calories Burned

AI Recommendation API

Endpoint:

```
POST /api/ai/workout-recommendation
{
  "age": 25,
  "fitnessGoal": "Muscle Gain",
  "experience": "Intermediate"
}
```

The intended response contains personalized workout guidance based on the submitted profile information.

AI Fitness Insights API

Endpoint:

```
POST /api/ai/fitness-insights
{
  "totalWorkouts": 10,
  "averageDuration": 45,
  "totalCaloriesBurned": 2500
}
```

The service is intended to return performance analysis, improvement suggestions, motivational guidance, and a progress summary.

Complete API Route Reference

Method	Endpoint	Auth	Purpose
GET	/api/health	No	Health/status check
POST	/api/auth/register	No	Register user

POST	/api/auth/login	No	Login user
GET	/api/auth/profile	Yes	Get profile
PUT	/api/auth/profile	Yes	Update profile
POST	/api/workouts	Yes	Create workout
GET	/api/workouts	Yes	List/filter workouts
GET	/api/workouts/search	Yes	Search workouts
GET	/api/workouts/:id	Yes	Get workout by ID
PUT	/api/workouts/:id	Yes	Update workout
DELETE	/api/workouts/:id	Yes	Delete workout
GET	/api/dashboard	Yes	Dashboard analytics
POST	/api/ai/workout-recommendation	Yes	AI recommendation
POST	/api/ai/fitness-insights	Yes	AI fitness insights

API Testing with Thunder Client

Thunder Client is the API testing tool used for this upgraded project. It runs inside Visual Studio Code and can test the same REST endpoints consumed by the React frontend.

- 1. GET /api/health
- 2. POST /api/auth/register
- 3. POST /api/auth/login
- 4. Copy the JWT token.
- 5. GET /api/auth/profile
- 6. PUT /api/auth/profile
- 7. POST /api/workouts
- 8. GET /api/workouts and search/filter requests
- 9. GET /api/workouts/:id
- 10. PUT /api/workouts/:id
- 11. GET /api/dashboard
- 12. POST AI recommendation and insight requests
- 13. DELETE /api/workouts/:id

Authentication API Testing

Register User

```
POST /api/auth/register
{
  "name": "John",
  "email": "john@example.com",
  "password": "12345678"
}
```

Login User

```
POST /api/auth/login
{
  "email": "john@example.com",
  "password": "12345678"
}
```

After login, copy the returned JWT token and use it as a Bearer token for protected requests.

Profile API Testing

Get Profile

```
GET /api/auth/profile
```

Update Profile

```
PUT /api/auth/profile
{
  "name": "John Alan",
  "age": 25,
  "heightCm": 175,
  "weightKg": 70,
  "fitnessGoal": "Muscle Gain",
  "experience": "Intermediate"
}
```

Workout API Testing - Read

Get All Workouts

```
GET /api/workouts?page=1&limit=10
```

Get Workout By ID

```
GET /api/workouts/:id
```

Search

```
GET /api/workouts/search?q=running
```

Workout API Testing - Update and Delete

Update Workout

```
PUT /api/workouts/:id
{
  "workoutName": "Morning Running - Updated",
  "duration": 40,
  "caloriesBurned": 330
}
```

Delete Workout

```
DELETE /api/workouts/:id
```

Dashboard API Testing

```
GET /api/dashboard
```

The response should include workout totals, average duration, streak information, category counts, weekly activity, and recent workouts.

AI API Testing and Current Gemini Access Note

Both AI endpoints are implemented in the backend and route through the Gemini service. During development, the project successfully connected to MongoDB Atlas and reached the Gemini service, but the Google project returned a 403 permission error indicating that the project had been denied access.

This is an external Google AI project access/configuration issue rather than a MongoDB, Express, or route-definition failure. A successful AI screenshot should be inserted after generation access is available. Until then, a controlled 403 screenshot may be retained as a troubleshooting record.

Environment Configuration and Execution

Sensitive configuration is stored in `server/.env`. The current application connects to MongoDB Atlas and runs the API on port 5000.

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@<cluster>/<database>?...
JWT_SECRET=<long-random-secret>
GEMINI_API_KEY=<google-gemini-api-key>
GEMINI_MODEL=gemini-3.6-flash
CLIENT_URL=http://localhost:5173
```

Backend

```
cd <project-folder>\server
npm install
npm run dev
```

Frontend

```
cd <project-folder>\client
npm install
npm run dev
```

Testing Checklist

- Backend starts without errors.
- MongoDB Atlas connection succeeds.
- Health endpoint returns success.
- Registration creates a user.
- Login returns a JWT.

- Profile GET and PUT work.
- Workout create, list, search, get-by-ID, update, and delete work.
- Dashboard returns statistics.
- React frontend loads successfully.
- Protected endpoints reject requests without valid authentication.
- AI endpoints are tested with an authorized Gemini project.
- No secrets are visible in screenshots.
- Thunder Client screenshots match the documented routes.

Error Handling and Security

The backend uses centralized error handling so route failures are returned as structured JSON rather than uncontrolled server output.

- Invalid credentials are rejected.
- Missing or invalid JWT tokens are rejected.
- Validation errors return controlled messages.
- Database failures are handled centrally.
- Not-found resources return appropriate errors.
- Gemini provider failures are logged and converted into controlled API errors.

Security Practices

- Passwords are hashed with bcryptjs.
- JWT protects authenticated resources.
- User-specific queries use the authenticated user ID.
- Secrets are stored in environment variables.
- Helmet and CORS are configured in Express.
- API keys, database passwords, and JWT secrets must not appear in screenshots or source control.

Future Enhancements and Conclusion

Future Enhancements

- Nutrition and meal tracking.
- BMI calculation.
- Wearable device integration.
- Workout reminders and scheduled notifications.
- Expanded analytics dashboards.
- Goal progress tracking.
- Additional AI coaching features.
- More granular roles and permissions.

Conclusion

AI FitTrack provides a structured full-stack solution for workout tracking and AI-assisted fitness guidance. The upgraded implementation combines React, Express, MongoDB Atlas, JWT authentication, workout CRUD, search and filters, dashboard analytics, and Gemini service integration.

The modular architecture supports maintainability and future growth. Thunder Client verifies each API independently while the React client demonstrates the same backend services in a user-facing application.

The project is ready for functional demonstration once the external Gemini project has the required generation access. The remaining authentication, workout, search, dashboard, and database functions operate independently of that external permission.

Documentation prepared from the supplied AI FitTrack reference document and the upgraded project implementation described in this project.